# Online Course Registration System (OCRS)

**Design, Prototype, Planning and Cost Estimation of a Software System**

Stamford University Bangladesh · Department of Computer Science & Engineering
Course: System Design & Software Engineering Sessional (0613CSE432) Faculty: Zerin Hasan Sahosh, Lecturer, Department of CSE Group: [Member 1 – Name, ID] · [Member 2 – Name, ID] · [Member 3 – Name, ID] Submission date: [DD Month YYYY]

## Chapter 1 – Introduction

### 1.1 Project Title

Design, Prototype, Planning and Cost Estimation of an Online Course Registration System (OCRS).

### 1.2 Background

At the start of every semester, university students must choose courses, check prerequisites, avoid class-time clashes, stay within the credit limit, get advisor approval and pay fees. In many universities this is still done through paper forms, spreadsheets, or loosely connected tools. Students queue to meet advisors, advisors sign forms by hand, and the registrar's office re-enters the data manually. Section seat limits are tracked informally, so seats are over-allocated or filled unfairly.

An online course registration system moves this workflow to a single web application with one source of truth for courses, seats, schedules, registrations and payments.

### 1.3 Problem Statement

The current manual or semi-manual registration process has these problems:

1. Registration is slow: students need several visits to advisors, the registrar and the accounts office.
2. Errors are common: missed prerequisites, overlapping class times and credit-limit violations are only found after registration.
3. Seat allocation is not reliable: capacity is tracked by hand, so sections are over-filled and full sections have no waiting mechanism.
4. Advisor approval is paper-based and hard to track.
5. Fee calculation and payment are disconnected from registration.
6. Administrators cannot easily see enrollment, seat utilization or payment status.

## 1.4 Proposed Solution

OCRS is a role-based web application with which:

- **Students** search offered courses, register, join waitlists, drop courses, pay fees online and view their schedule.
- **Faculty Advisors** review and approve or reject their advisees' registrations online.
- **Administrators** manage courses, offerings, seat capacity, registration dates, users and reports.

The system automatically validates prerequisites, schedule conflicts, seat availability and credit limits, and sends notifications for important events.

## 1.5 Objectives

1. Reduce the time a student needs to complete registration to under 5 minutes.
2. Eliminate manual validation errors (prerequisites, time conflicts, credit overload, seat overbooking).
3. Provide an online advisor approval workflow with remarks and a status trail.
4. Link registration with fee invoicing and online payment.
5. Give administrators real-time reports on enrollment, seat utilization and payments.
6. Apply the Scrum process model, UML design, formal estimation (Use Case Points), testing and traceability to the project, as required by the course.

## 1.6 Target Users

| User | Description |
|---|---|
| Student | Enrolled undergraduate student who registers each semester |
| Faculty Advisor | Teacher assigned to guide and approve a group of students |
| Administrator | Registrar's office / department staff who manage academic data |

## 1.7 Project Scope

**The system will:**

- Authenticate users and enforce role-based access.
- Manage courses, prerequisites, semesters, offerings (sections), time slots and seat capacity.

- Let students search, register, drop and waitlist for courses with automatic validation.
- Support advisor approval or rejection of registrations.
- Generate a fee invoice and record online payment through a payment gateway.
- Send email/SMS notifications, produce a registration slip (PDF) and generate reports.

**The system will not:**

- Manage grades, results, transcripts or CGPA calculation (it only reads 'completed courses' data for prerequisite checks).
- Manage faculty payroll, library, hostel, attendance or examination scheduling.
- Provide a native mobile app (the web interface is mobile-responsive only).
- Handle multi-campus or multi-university deployment.

### 1.8 Assumptions

- Students' completed-course history is available (entered by Admin or imported as CSV).
- Every student is assigned to exactly one faculty advisor.
- A third-party payment gateway (e.g., SSLCommerz or bKash sandbox) is available for fee payment.
- The registration, add/drop and payment dates are set by the university each semester.

## Chapter 2 – Requirement Analysis

### 2.1 Actors

| Actor | Type | Description |
|---|---|---|
| Student | Primary | Registers, drops, pays, views schedule |
| Faculty Advisor | Primary | Reviews and approves/rejects registrations of advisees |
| Administrator | Primary | Manages courses, offerings, users, reports |
| Payment Gateway | Secondary (system) | External system that processes online payments |
| Notification Service (Email/SMS) | Secondary (system) | External service that delivers notifications |

Student, Faculty Advisor and Administrator are specializations of the generalized actor **User**, who can log in and manage his/her own profile.

## 2.2 User Roles

| Role | Permissions |
|---|---|
| Student | Own profile, course catalog (read), own registrations, waitlist, drop, payments, schedule |
| Faculty Advisor | Advisee list (read), registration review (approve/reject with remarks), course catalog (read) |
| Administrator | Full CRUD on courses, offerings, semesters, users and roles; reports; override operations |

## 2.3 Functional Requirements

| ID | Requirement (testable statement) | Priority |
|---|---|---|
| FR01 | The system shall authenticate a user by user ID and password and open the dashboard of the user's role; after 5 consecutive failed attempts the account shall be locked for 15 minutes. | Must |
| FR02 | The system shall allow a user to view and update contact details and change the password. | Should |
| FR03 | The system shall allow a student to search and filter offered courses by code, title, department, semester and instructor. | Must |
| FR04 | The system shall allow a student to register for an offered course section during the registration period. | Must |
| FR05 | The system shall reject a registration if the student has not completed all prerequisites of the course and shall display the missing prerequisites. | Must |
| FR06 | The system shall reject a registration whose class time overlaps with a course the student has already registered for in the same semester. | Must |

| ID | Requirement (testable statement) | Priority |
|---|---|---|
| FR07 | The system shall reject a registration when the section has no free seat or when the student's total credits would exceed the semester credit limit (default 18). | Must |
| FR08 | The system shall add a student to a waitlist when a section is full and shall automatically offer the seat to the first waitlisted student when a seat is released. | Should |
| FR09 | The system shall allow a student to drop a registered course before the drop deadline and shall release the seat immediately. | Must |
| FR10 | The system shall allow a faculty advisor to approve or reject a pending registration of an advisee with a remark. | Must |
| FR11 | The system shall generate a fee invoice from the registered credits and shall record the payment status returned by the payment gateway. | Must |
| FR12 | The system shall display the student's weekly class schedule and the status (Pending, Approved, Rejected, Waitlisted, Dropped) of each registration. | Must |
| FR13 | The system shall allow an administrator to create, update and deactivate courses, prerequisites, offerings, time slots, seat capacity and registration/drop dates. | Must |
| FR14 | The system shall allow an administrator to create, update, deactivate users and assign roles. | Must |
| FR15 | The system shall send an email/SMS notification on registration submission, approval/rejection, waitlist promotion and payment confirmation. | Should |
| FR16 | The system shall generate a printable registration slip (PDF) containing approved courses, schedule and payment status. | Could |
| FR17 | The system shall allow an administrator to generate course enrollment, seat utilization and payment summary reports. | Should |

## 2.4 Non-Functional Requirements

| ID | Category | Requirement | Measure |
|---|---|---|---|
| NFR01 | Performance | Pages shall load and registration requests complete quickly even in the registration rush. | 95% of requests ⩽ 3 s with 500 concurrent users |
| NFR02 | Security | Passwords stored hashed (bcrypt); all traffic over HTTPS; role-based authorization on every request; sessions expire when idle. | 0 pages accessible without authorization; 15-min idle timeout |
| NFR03 | Availability | The system shall be available during registration windows. | ⩾ 99% uptime in registration periods |
| NFR04 | Usability | A first-time student shall register without training; UI responsive on phone, tablet, desktop; supports latest Chrome, Firefox, Edge. | Task completed ⩽ 5 min by 90% of test users |
| NFR05 | Reliability / Data integrity | Seat counts must never be over-allocated, even with simultaneous requests; critical operations are transactional; daily backups. | 0 over-booked seats in concurrency test; RPO ⩽ 24 h |
| NFR06 | Maintainability | Layered, modular architecture with documented code. | Unit-test coverage ⩾ 70% of business logic |
| NFR07 | Scalability | The system shall support growth in users without redesign. | Up to 5,000 students with horizontal scaling |

## 2.5 System Requirements

| Item | Requirement |
|---|---|
| Client | Any modern browser (Chrome, Firefox, Edge) on PC, tablet or phone; internet connection |
| Server | 2 vCPU, 4 GB RAM, 40 GB SSD (Linux VPS) – scalable |
| Back end / Database | Web framework (e.g., Node.js/Express, Django or Spring Boot) with a relational DBMS (PostgreSQL or MySQL) |
| External services | Payment gateway sandbox/API; SMTP email service; optional SMS API |
| Security | TLS certificate (free via Let's Encrypt) |

## 2.6 Major Modules

| Module | Main features |
|---|---|
| M1 – User & Access Management | Login, profile, user/role management, authentication, RBAC |
| M2 – Course & Offering Management | Courses, prerequisites, semesters, sections, time slots, seats, dates |
| M3 – Registration Management | Search, register with validation, drop, waitlist, advisor approval, schedule |
| M4 – Payment & Billing | Invoice generation, gateway payment, receipt/registration slip |
| M5 – Notification & Reporting | Email/SMS alerts, enrollment/seat/payment reports |

## 2.7 Major Business Processes

1. **Course Registration Process** – search → select section → automatic validation → submit → advisor approval → confirmation.
2. **Fee Payment Process** – invoice generation → payment via gateway → status update → receipt.

3. **Drop & Waitlist Promotion Process** – drop course → release seat → promote first waitlisted student → notify.

### 2.8 Compliance with Handbook Minimums

| Handbook minimum | OCRS |
|---|---|
| ⩾ 3 major actors | 3 primary + 2 secondary |
| ⩾ 8 use cases | 14 user-goal + 4 included use cases |
| ⩾ 10 functional requirements | 17 |
| ⩾ 5 non-functional requirements | 7 |
| ⩾ 3 modules | 5 |
| ⩾ 2 business processes | 3 |
| ⩾ 2 user roles | 3 |
| Database functionality | Relational database (see class diagram) |
| Authentication / RBAC | FR01, NFR02 |

## Chapter 3 – Software Process Model

### 3.1 Selected Model: Scrum

Scrum is an Agile framework that builds the product in fixed-length iterations (sprints) of 1–4 weeks. Each sprint takes the highest-priority items from the Product Backlog and delivers a potentially shippable increment. Scrum has three roles (Product Owner, Scrum Master, Developers) and five events (Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective).

### 3.2 Why Scrum Was Selected

- **Natural fit with features.** OCRS splits cleanly into independent user stories (login, catalog, register, approve, pay, report), so each sprint can deliver a working feature.
- **Requirements are likely to change.** Registration rules (credit limit, drop deadlines, waitlist policy, payment methods) differ between departments and change every semester. Scrum accepts reordering of the backlog at each sprint.
- **Early feedback.** Students, advisors and admin staff can try the increment after every sprint, which reduces the risk of building the wrong workflow.

- **Risk-driven ordering.** The riskiest parts (validation logic and seat concurrency) are built in the middle sprints, with enough time left to fix them.
- **Small team.** A team of a few people with continuous communication suits Scrum's lightweight ceremonies.

## 3.3 Why Other Models Were Not Chosen

| Model | Reason not selected |
|---|---|
| Waterfall | Late testing and no room for changing registration policies; users see the system only at the end |
| Incremental | Good, but has no built-in inspect-and-adapt cycle or backlog re-prioritization |
| Iterative / Spiral | Spiral's heavy risk analysis is more than a small, well-understood domain needs |
| Prototyping | Useful for the UI, but we also need a complete, tested working system; we use prototyping only inside Sprint 0 |

If Waterfall were used instead, all requirements would be frozen in weeks 1–3, testing would only start after coding, and a change in the credit-limit policy would require a formal change request and rework of design and test documents.

## 3.4 How Scrum Is Applied to OCRS

| Scrum element | Application in OCRS |
|---|---|
| Sprint length | 2 weeks (Sunday–Thursday); Sprint 0 for setup, then 6 delivery sprints, then a release sprint |
| Product Owner | Represents students, advisors and registrar; owns and orders the Product Backlog |
| Scrum Master | Facilitates events, removes blockers, protects the team |
| Developers | Analyst, UI/UX designer, developers, DB engineer |
| Testers / QA | Test each story in-sprint; own the Definition of Done for testing |
| Sprint Planning | Start of each sprint: select stories up to the team's velocity (≈ 20 story points) |

| Scrum element | Application in OCRS |
|---|---|
| Daily Scrum | 15 minutes daily: done yesterday, plan today, blockers |
| Sprint Review | Demo the increment to the Product Owner and sample users |
| Retrospective | Improve process (e.g., code review rules, estimation accuracy) |
| Definition of Done | Code reviewed, unit tests pass, acceptance criteria verified by QA, merged to main, deployed to staging, documentation updated |

## 3.5 Product Backlog

Story points use the Fibonacci scale. Priority uses MoSCoW.

| ID | User story | FR | Priority | Points | Sprint | Acceptance criteria |
|---|---|---|---|---|---|---|
| PB01 | As a user, I want to log in so that I see the dashboard of my role. | FR01 | Must | 5 | 1 | Valid credentials open the correct dashboard; invalid ones show an error; 5 failed attempts lock the account |
| PB02 | As a user, I want to update my profile and password. | FR02 | Should | 3 | 1 | Changes saved and visible; old password required to change password |
| PB03 | As an admin, I want to manage courses and prerequisites. | FR13 | Must | 8 | 2 | Course can be created, edited, deactivated; prerequisite links saved; duplicates by code rejected |
| PB04 | As an admin, I want to manage offerings, time slots, seats and registration dates. | FR13 | Must | 8 | 2 | Section has instructor, time slot, capacity; registration window enforced |
| PB05 | As a student, I want to search and filter courses. | FR03 | Must | 5 | 2 | Search by code/title/department returns matching offerings within 3 s |
| PB06 | As a student, I want to register for a course and get automatic validation. | FR04–FR07 | Must | 13 | 3 | Missing prerequisite, time clash, full section and credit overload are each rejected with a clear message; valid registration saved as Pending |

| ID | User story | FR | Priority | Points | Sprint | Acceptance criteria |
|---|---|---|---|---|---|---|
| PB07 | As a student, I want to see my weekly schedule and registration status. | FR12 | Must | 5 | 3 | Timetable shows all registered sections with status |
| PB08 | As an advisor, I want to approve or reject advisees' registrations. | FR10 | Must | 8 | 4 | Only own advisees are visible; remark required on rejection; status updates immediately |
| PB09 | As a student, I want to drop a course before the deadline. | FR09 | Must | 5 | 4 | Drop allowed before deadline only; seat released immediately |
| PB10 | As a student, I want to join a waitlist for a full section. | FR08 | Should | 8 | 4 | Waitlist position shown; first student is auto-enrolled when a seat is released |
| PB11 | As a student, I want an invoice and to pay online. | FR11 | Must | 13 | 5 | Invoice equals credits × fee rate; successful payment marked Paid; failure leaves Unpaid |
| PB12 | As a user, I want notifications on important events. | FR15 | Should | 5 | 5 | Email sent for registration, approval/rejection, waitlist promotion, payment |
| PB13 | As a student, I want to print my registration slip. | FR16 | Could | 3 | 5 | PDF contains student info, approved courses, schedule and payment status |
| PB14 | As an admin, I want to manage user accounts and roles. | FR14 | Must | 5 | 6 | User created/disabled; role change takes effect on next login |
| PB15 | As an admin, I want enrollment, seat and payment reports. | FR17 | Should | 8 | 6 | Reports filter by semester and department; export to CSV/PDF |

**Total backlog = 102 story points.**

## 3.6 Sprint Plan

| Sprint | Weeks | Goal | Backlog items | Points |
|---|---|---|---|---|
| Sprint 0 | 1–2 | Project setup: requirement finalization, backlog, architecture, database schema, UI wireframes, repository and CI | – (setup tasks) | – |
| Sprint 1 | 3–4 | Users can log in securely and manage profiles | PB01, PB02 | 8 |
| Sprint 2 | 5–6 | Admin builds the catalog; students can find courses | PB03, PB04, PB05 | 21 |

| Sprint | Weeks | Goal | Backlog items | Points |
|---|---|---|---|---|
| Sprint 3 | 7–8 | Students can register with full validation and see their schedule (highest-risk sprint) | PB06, PB07 | 18 |
| Sprint 4 | 9–10 | Advisor approval, drop and waitlist workflows | PB08, PB09, PB10 | 21 |
| Sprint 5 | 11–12 | Payment, notifications, registration slip | PB11, PB12, PB13 | 21 |
| Sprint 6 | 13–14 | User management, reports, security hardening, performance tuning | PB14, PB15 | 13 |
| Release | 15–16 | System testing, user acceptance testing, bug fixing, deployment and handover | – | – |

Sprint 1 is lighter because the team is still learning the stack and setting up the pipeline; Sprint 6 is lighter because it includes hardening and load tests. The plan averages about 17 points per delivery sprint (102 ÷ 6).

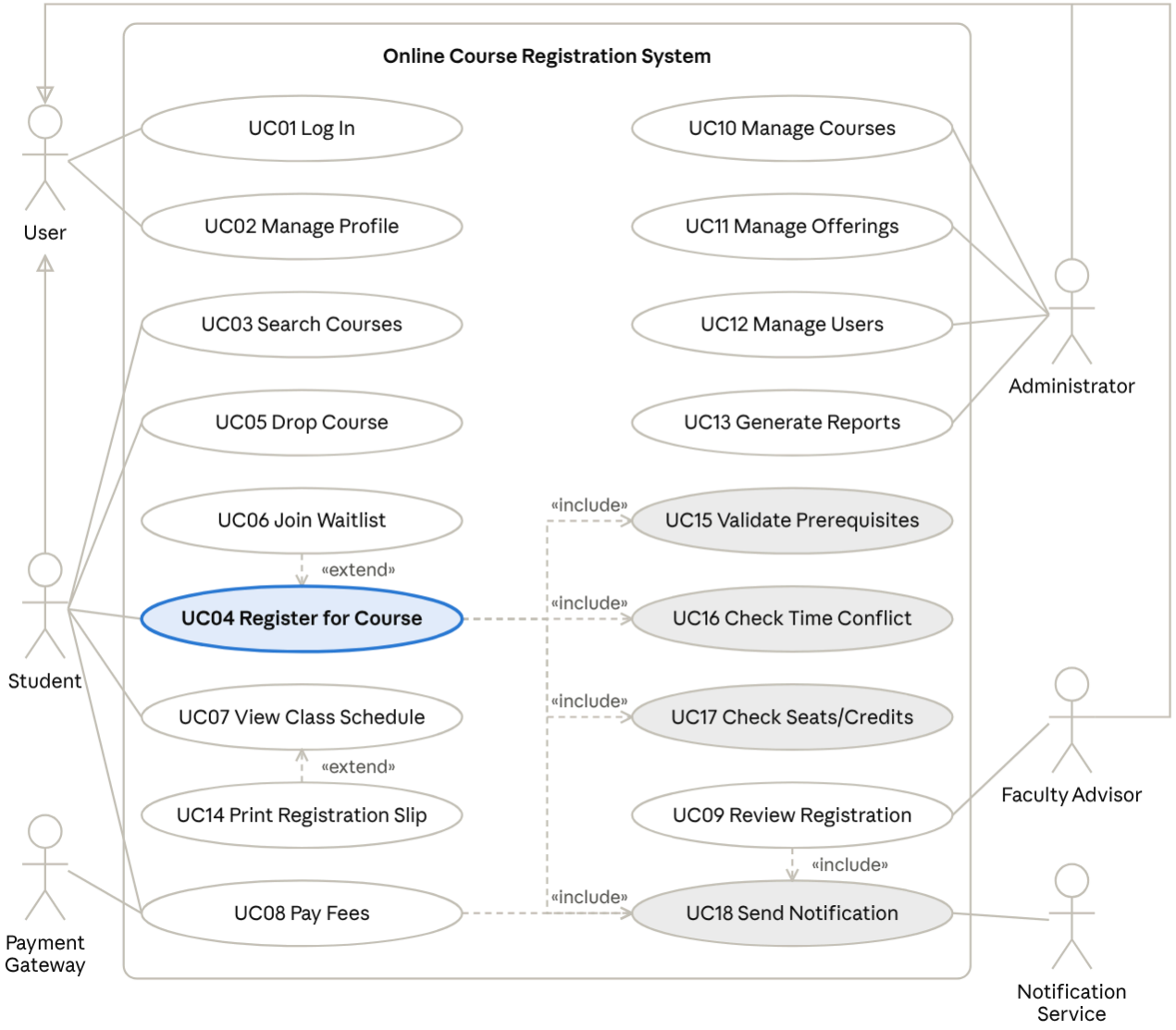
# Chapter 4 – UML Design

All diagrams follow UML 2 notation and reuse the same identifiers (FR, UC, class names) so that they can be traced in Chapter 10.

## 4.1 Use Case Diagram

Inside the system boundary are 14 user-goal use cases and 4 included sub-functions; outside it are 3 primary and 2 secondary actors.

**Register for Course is the hub use case of OCRS**




use case diagram · 5 actors, 18 use cases

UC04 (highlighted) includes the three checks UC15–UC17 and the notification step UC18 (shaded); UC06 and UC14 extend it. Student, Faculty Advisor and Administrator generalize User.

## Use case list

| ID | Use case | Actor(s) | FR |
|---|---|---|---|
| UC01 | Log In | User (Student, Advisor, Admin) | FR01 |
| UC02 | Manage Profile | User | FR02 |
| UC03 | Search Courses | Student | FR03 |
| UC04 | Register for Course | Student | FR04 |

| ID | Use case | Actor(s) | FR |
|---|---|---|---|
| UC05 | Drop Course | Student | FR09 |
| UC06 | Join Waitlist | Student | FR08 |
| UC07 | View Class Schedule | Student | FR12 |
| UC08 | Pay Fees | Student; Payment Gateway | FR11 |
| UC09 | Review Registration | Faculty Advisor | FR10 |
| UC10 | Manage Courses | Administrator | FR13 |
| UC11 | Manage Offerings | Administrator | FR13 |
| UC12 | Manage Users | Administrator | FR14 |
| UC13 | Generate Reports | Administrator | FR17 |
| UC14 | Print Registration Slip | Student | FR16 |
| UC15 | Validate Prerequisites (included) | – | FR05 |
| UC16 | Check Time Conflict (included) | – | FR06 |
| UC17 | Check Seats/Credits (included) | – | FR07 |
| UC18 | Send Notification (included) | Notification Service | FR15 |

### Relationships and why they were used

| Relationship | Reason |
|---|---|
| UC04 «include» UC15, UC16, UC17 | Every registration must pass these three checks; they are mandatory, reusable steps of UC04 |
| UC04, UC08, UC09 «include» UC18 | Each of these always ends by notifying a user, so the notification step is shared |
| UC06 «extend» UC04 | Joining a waitlist happens only when the seat check fails (optional, conditional behaviour) |
| UC14 «extend» UC07 | Printing a slip is an optional action the student may take while viewing the schedule |

| Relationship | Reason |
| --- | --- |
| Student, Faculty Advisor, Administrator generalize User | All three log in and manage a profile (UC01, UC02), so those use cases belong to the general actor |

### Major use case descriptions

UC04 – Register for Course

| Field | Description |
| --- | --- |
| Primary actor | Student |
| Precondition | Student is logged in; the registration window is open; the section is offered |
| Main flow | 1. Student selects an offered section. 2. System runs UC15, UC16 and UC17. 3. System saves the registration as Pending and reserves one seat. 4. System runs UC18 to notify the student and the advisor. 5. System shows a confirmation. |
| Alternate flows | 2a. A check fails: the system shows the reason and creates no registration. 2b. The section is full: the system offers UC06 (Join Waitlist). |
| Postcondition | A Pending registration exists and the seat count is updated |

UC05 – Drop Course

| Field | Description |
| --- | --- |
| Primary actor | Student |
| Precondition | The student has a Pending or Approved registration; the drop deadline has not passed |
| Main flow | 1. Student selects the registration and confirms the drop. 2. System sets the status to Dropped and releases the seat. 3. System recalculates the invoice. 4. System offers the seat to the first waitlisted student through the UC04 save-and-notify steps. |

| Field | Description |
|---|---|
| Alternate flow | 1a. The deadline has passed: the system refuses the drop and shows the deadline. |
| Postcondition | Seat released or re-assigned; invoice updated |

### UC08 – Pay Fees

| Field | Description |
|---|---|
| Primary actor | Student (secondary actor: Payment Gateway) |
| Precondition | At least one registration is Approved and the invoice is Unpaid |
| Main flow | 1. System displays the invoice (credits × fee rate). 2. Student chooses a payment method. 3. System redirects to the gateway. 4. Gateway returns the result. 5. System records the payment and marks the invoice Paid. 6. System runs UC18 to send a receipt message. |
| Alternate flow | 4a. Payment fails or is cancelled: the invoice stays Unpaid and the student may retry. |
| Postcondition | Invoice Paid and a Payment record saved |

### UC09 – Review Registration

| Field | Description |
|---|---|
| Primary actor | Faculty Advisor |
| Precondition | Advisor is logged in; an advisee has a Pending registration |
| Main flow | 1. Advisor opens the pending list of advisees. 2. Advisor selects a registration. 3. Advisor chooses Approve or Reject (a remark is mandatory for Reject). 4. System updates the status; on Reject it releases the seat. 5. System runs UC18 to notify the student. |
| Alternate flow | 2a. The student is not an advisee: access is denied. |
| Postcondition | Registration is Approved or Rejected |

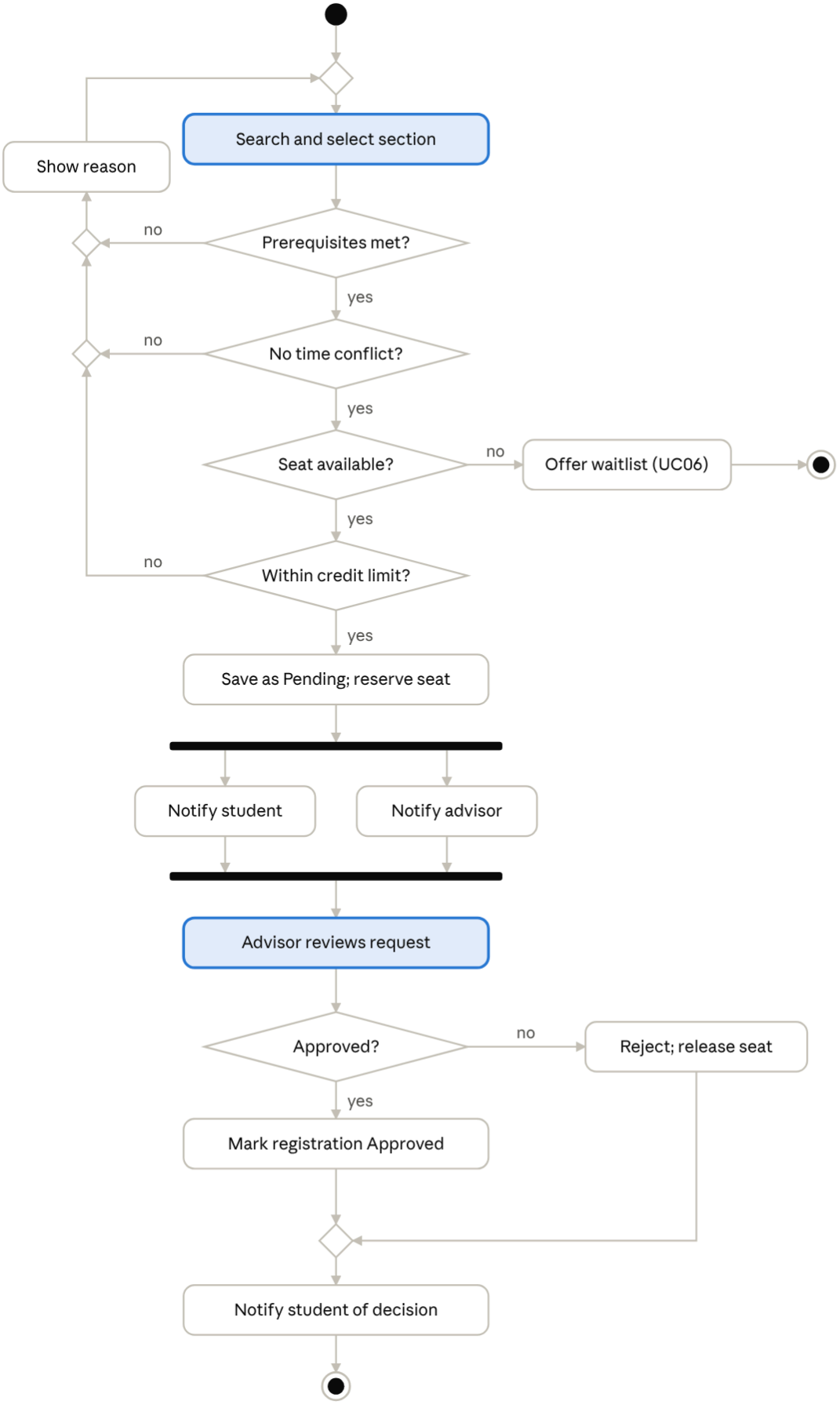
UC11 – Manage Offerings

| Field | Description |
|---|---|
| Primary actor | Administrator |
| Precondition | Admin is logged in; the course exists |
| Main flow | 1. Admin selects a course and a semester. 2. Admin enters section, instructor, time slot, capacity, registration and drop dates. 3. System validates the data (no instructor clash, capacity > 0). 4. System saves the offering and shows it in the catalog. |
| Alternate flows | 3a. Data invalid: the system shows the errors. 4a. Deactivating an offering that has registrations: the system warns and asks for confirmation. |
| Postcondition | The offering is available for registration |

## 4.2 Activity Diagrams

Two important processes are modelled: AD1 Course Registration and Approval (UC04, UC09) and AD2 Fee Payment (UC08).

### AD1: any failed check loops back; approval ends the process

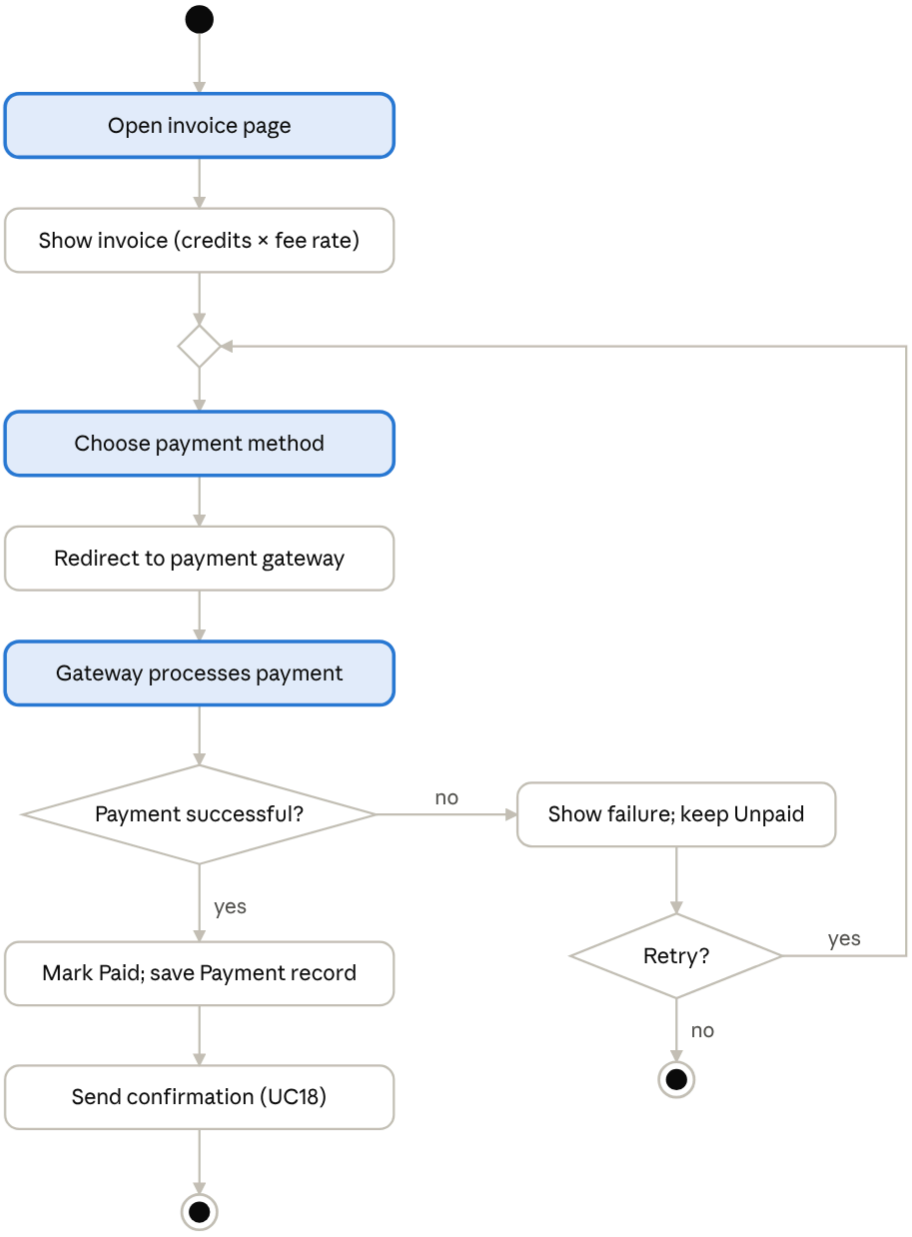



Shaded = performed by an actor (Student, Advisor); plain = performed by the system

AD1 activity diagram · 5 decisions, 1 fork/join, 2 final nodes

AD1 covers UC04, UC06, UC09, UC15–UC18: a failed check returns the student to course selection, a full section ends in a waitlist offer, the fork sends both notifications in parallel, and a rejection releases the seat.

**AD2: a failed payment keeps the invoice Unpaid and allows a retry**



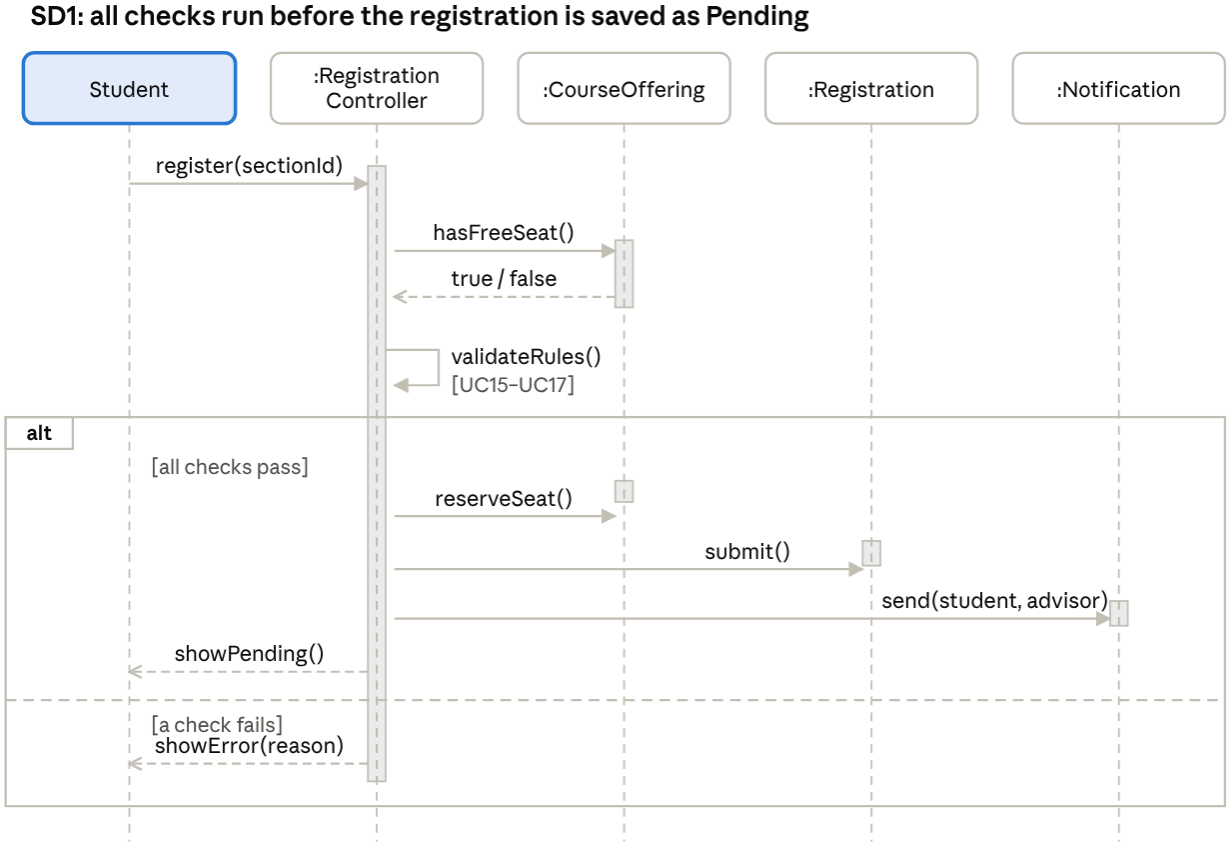

Shaded = performed by an actor (Student, Payment Gateway); plain = performed by the system

AD2 activity diagram · 2 decisions, retry loop

AD2 covers UC08 and UC18: the retry loop returns to payment-method selection, and success ends with the confirmation message.

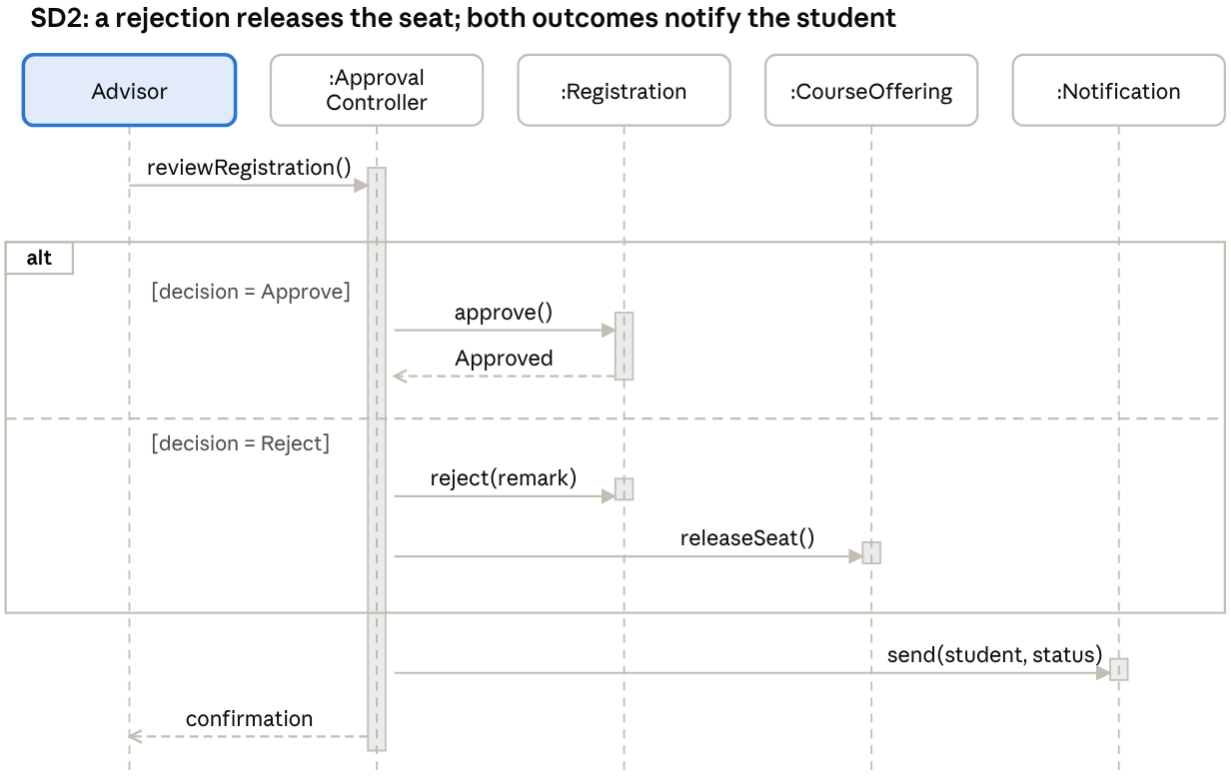
## 4.3 Sequence Diagrams

Two scenarios are modelled: SD1 Register for Course (UC04) and SD2 Approve a Registration (UC09). Messages sent to domain objects (reserveSeat, submit, approve, reject, releaseSeat, send) match operations in the class diagram; the controller object belongs to the design layer.




SD1 sequence diagram · 5 lifelines, 1 alt frame

SD1 realizes UC04: the controller checks the seat and the rules, then (only if they pass) reserves the seat, creates the Pending registration and sends the notification (UC18). If a check fails the student gets the reason and may be offered the waitlist (UC06).



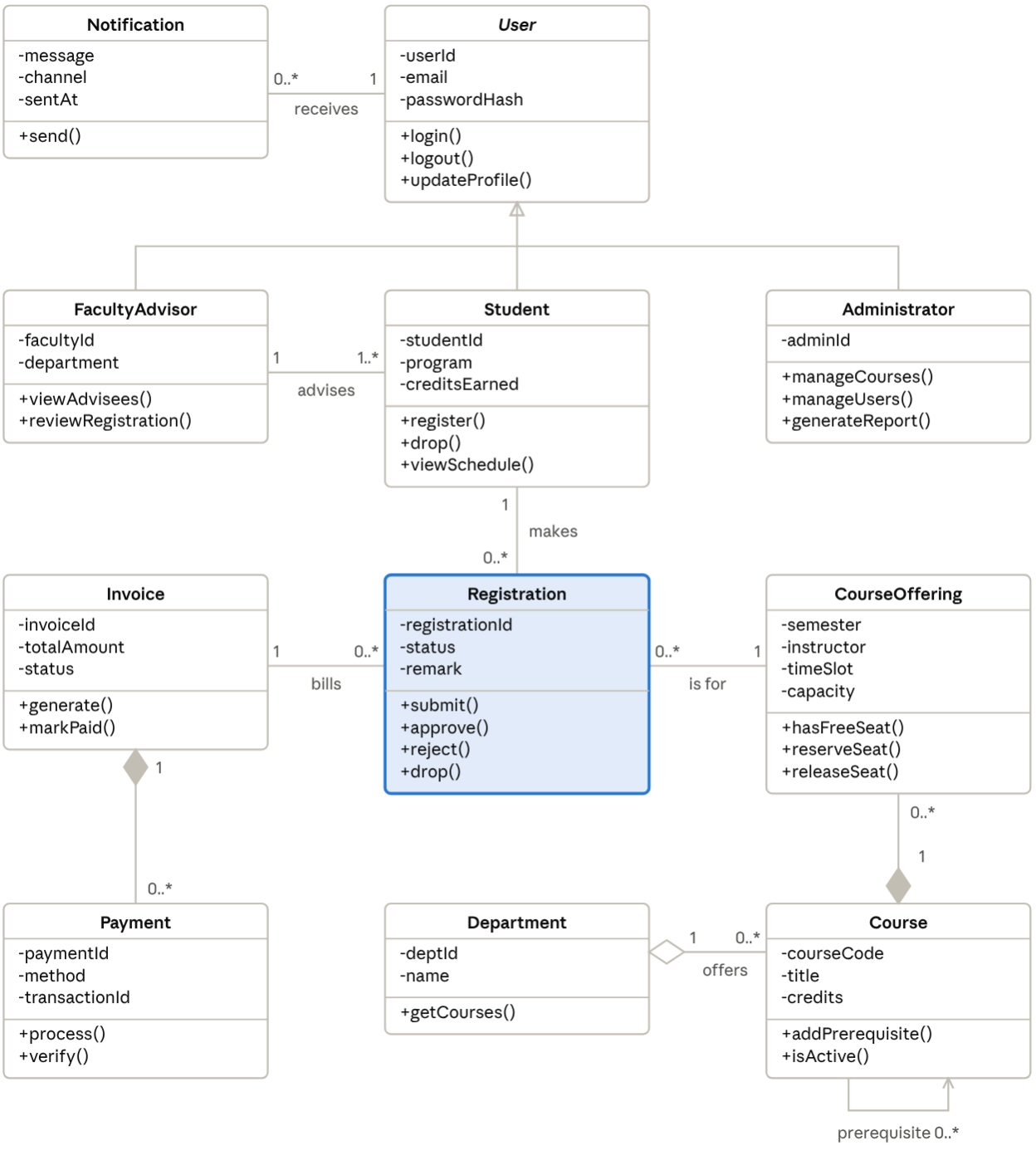

SD2 sequence diagram · 5 lifelines, 1 alt frame

SD2 realizes UC09: approval only updates the status, while rejection also releases the seat of the offering; the student is notified in both cases (UC18).

## 4.4 Class Diagram

The class diagram shows 11 classes, generalization (User), composition (Course–CourseOffering, Invoice–Payment), aggregation (Department–Course) and multiplicities.

**OCRS domain model: Registration links students, offerings and invoices**




class diagram · 11 classes, 1 generalization, 2 compositions, 1 aggregation

Reading guide: User is abstract (italic) and is specialized by FacultyAdvisor, Student and Administrator; filled diamonds mark composition (an offering or a payment cannot exist without its course or invoice); the hollow diamond marks aggregation (a department groups courses that exist on their own); the highlighted Registration class is the core of UC04, UC05, UC06 and UC09.

### Why each class is required

| Class | Why it is needed |
|---|---|
| User (abstract) | Holds login and profile data shared by all roles |
| Student | Registers and drops courses; owns registrations (UC03–UC08, UC14) |
| FacultyAdvisor | Reviews the registrations of assigned students (UC09) |
| Administrator | Manages courses, offerings, users and reports (UC10–UC13) |
| Department | Groups courses; supports search and reports by department |
| Course | Catalog entry with code, title, credits and prerequisites |
| CourseOffering | A course in a semester with instructor, time slot and capacity; basis for the seat and conflict checks |
| Registration | Links a student to an offering with a status; core of UC04, UC05, UC06 and UC09 |
| Invoice | Fee amount for a student's approved registrations (UC08) |
| Payment | One gateway transaction against an invoice |
| Notification | A message sent to a user by UC18 |

# Chapter 5 – Prototype

## 5.1 Overview

The OCRS prototype is a clickable, high-fidelity model of the web interface (desktop and mobile layouts) that demonstrates login, role-based dashboards, course search, registration with validation messages, advisor approval, payment and admin management. The recommended tool is Figma (free student plan); HTML/CSS/JavaScript is an accepted alternative. This chapter specifies what the prototype must show.

Prototype link: [insert Figma / GitHub Pages link]

## 5.2 Screen Inventory

| ID | Screen | Role | Use case | FR | Key elements |
|---|---|---|---|---|---|
| S01 | Login | All | UC01 | FR01 | User ID, password, error message, locked-account message |
| S02 | My Profile | All | UC02 | FR02 | Contact details form, change-password form |
| S03 | Student Dashboard | Student | UC07 | FR12 | Registration status summary, deadlines, quick links |
| S04 | Course Search | Student | UC03 | FR03 | Search box, filters (department, semester, instructor), results with seats left |
| S05 | Register for Course | Student | UC04, UC15–UC17 | FR04–FR07 | Selected sections, credit counter, inline messages for prerequisite, time conflict, full section, credit limit |
| S06 | Waitlist Dialog | Student | UC06 | FR08 | Full-section notice, join waitlist button, waitlist position |
| S07 | My Schedule & Status | Student | UC07 | FR12 | Weekly timetable, status badges, Print Slip button |
| S08 | Drop Course Dialog | Student | UC05 | FR09 | Confirmation, deadline notice |
| S09 | Invoice & Payment | Student | UC08 | FR11 | Invoice lines, total, payment method, gateway redirect, result page |
| S10 | Pending Approvals | Advisor | UC09 | FR10 | Advisee list, registration details, Approve/Reject with remark |
| S11 | Manage Courses | Admin | UC10 | FR13 | Course table, add/edit form, prerequisite selector |
| S12 | Manage Offerings | Admin | UC11 | FR13 | Section form: instructor, time slot, capacity, dates |
| S13 | Manage Users | Admin | UC12 | FR14 | User table, create/disable, role dropdown |
| S14 | Reports | Admin | UC13 | FR17 | Report type and filters, table/chart, CSV export |
| S15 | Registration Slip | Student | UC14 | FR16 | Printable PDF preview |
| S16 | Notifications | All | UC18 | FR15 | Notification list/toast and sample email text |

## 5.3 Navigation

- **Student:** S01 → S03 → S04 → S05 (→ S06 if full) → S07 (→ S08 drop, S15 slip) → S09.
- **Advisor:** S01 → S10 (approve/reject dialog) → S16.

- **Admin:** S01 → Admin home → S11 / S12 / S13 / S14.
- **All roles:** S02 and S16 are reachable from the top navigation bar; every page has a logout link.

### 5.4 Demonstration Workflows

1. **End-to-end registration:** student searches and registers (S04, S05) → advisor approves (S10) → student sees Approved (S07) → student pays (S09) → prints slip (S15).
2. **Validation:** student tries a course with a missing prerequisite, a time clash and a full section; the waitlist dialog (S06) appears for the full section.
3. **Administration:** admin creates an offering (S12) and checks the enrollment report (S14).

### 5.5 Design Guidelines and Screenshots

Status badges use one colour per state (Pending amber, Approved green, Rejected red, Waitlisted blue, Dropped grey). Validation messages appear next to the action that caused them. Layouts are responsive and keyboard-accessible.

[Insert screenshots of S01–S16 here, each with a one-line caption.]

## Chapter 6 – Estimation

The estimated effort is **16.0 person-months (2,556 person-hours)** and the estimated cost is **৳1,155,000**, for a 16-week project with a team of 9 specialists working part-time (average ≈ 4.3 full-time equivalents).

### 6.1 Method

The formal technique is **Use Case Point (UCP) estimation** (Karner). It was chosen because OCRS is fully described by use cases, so the size can be counted directly from the UML artifacts of Chapter 4 at an early stage. The steps are: classify actors (UAW), classify use cases (UUCW), apply technical (TCF) and environmental (ECF) factors, then convert UCP to effort.

### 6.2 Software Size

**Step 1 – Unadjusted Actor Weight (UAW)**

| Actor | Class | Weight | Reason |
|---|---|---|---|
| Student | Complex | 3 | Graphical user interface |

| Actor | Class | Weight | Reason |
|---|---|---|---|
| Faculty Advisor | Complex | 3 | Graphical user interface |
| Administrator | Complex | 3 | Graphical user interface |
| Payment Gateway | Simple | 1 | External system through an API |
| Notification Service | Simple | 1 | External system through an API |
| **UAW** | | **11** | |

### Step 2 – Unadjusted Use Case Weight (UUCW)

| Class | Transactions | Weight | Use cases | Count | Total |
|---|---|---|---|---|---|
| Simple | ⩽ 3 | 5 | UC01, UC02, UC03, UC06, UC07, UC14 | 6 | 30 |
| Average | 4–7 | 10 | UC05, UC08, UC09, UC10, UC11, UC12, UC13 | 7 | 70 |
| Complex | > 7 | 15 | UC04 | 1 | 15 |
| **UUCW** | | | | 14 | **115** |

Only the 14 user-goal use cases are counted. UC15–UC18 are included sub-functions; their transactions are already counted in the parent use cases, which is why UC04 is classed as Complex.

```
UUCP = UAW + UUCW = 11 + 115 = 126
```

### Step 3 – Technical Complexity Factor (TCF)

| Factor | Description | Weight | Rating (0–5) | Weight × rating |
|---|---|---|---|---|
| T1 | Distributed (web) system | 2 | 3 | 6.0 |
| T2 | Response/throughput objectives | 1 | 4 | 4.0 |
| T3 | End-user efficiency | 1 | 4 | 4.0 |
| T4 | Complex internal processing | 1 | 2 | 2.0 |
| T5 | Reusable code | 1 | 2 | 2.0 |
| T6 | Easy to install | 0.5 | 2 | 1.0 |
| T7 | Easy to use | 0.5 | 5 | 2.5 |
| T8 | Portable | 2 | 3 | 6.0 |
| T9 | Easy to change | 1 | 3 | 3.0 |

| Factor | Description | Weight | Rating (0–5) | Weight × rating |
|---|---|---|---|---|
| T10 | Concurrent use | 1 | 4 | 4.0 |
| T11 | Special security objectives | 1 | 4 | 4.0 |
| T12 | Direct access for third parties | 1 | 3 | 3.0 |
| T13 | Special user training | 1 | 2 | 2.0 |
| **Total** | | | | **43.5** |

$$TCF = 0.6 + 0.01 \times 43.5 = 1.035$$

### Step 4 – Environmental Complexity Factor (ECF)

| Factor | Description | Weight | Rating (0–5) | Weight × rating |
|---|---|---|---|---|
| E1 | Familiarity with the process (Scrum) | 1.5 | 3 | 4.5 |
| E2 | Application experience | 0.5 | 2 | 1.0 |
| E3 | Object-oriented experience | 1 | 3 | 3.0 |
| E4 | Lead analyst capability | 0.5 | 3 | 1.5 |
| E5 | Motivation | 1 | 4 | 4.0 |
| E6 | Stable requirements | 2 | 3 | 6.0 |
| E7 | Part-time staff | −1 | 4 | −4.0 |
| E8 | Difficult programming language | −1 | 2 | −2.0 |
| **Total** | | | | **14.0** |

$$ECF = 1.4 - 0.03 \times 14.0 = 0.98$$

### Step 5 – Use Case Points

$$UCP = UUCP \times TCF \times ECF = 126 \times 1.035 \times 0.98 = 127.8$$

## 6.3 Effort

Karner's productivity is 20 person-hours per UCP. The Schneider–Winters rule confirms this value: only E2 (rated below 3) and E7 (rated above 3) count against the team, giving 2, and a count of 2 or less keeps 20 hours per UCP.

```
Effort = 127.8 \times 20 = 2{,}556 \text{ person-hours} = 2{,}556 / 160 =
16.0 \text{ person-months}
```

## 6.4 Development Time and Team Size

The project is time-boxed to 16 weeks (3.7 months): Sprint 0, six two-week delivery sprints and a two-week release phase. Average staffing is 16.0 person-months ÷ 3.7 months ≈ **4.3 full-time equivalents**. Because specialists are needed for different periods, the team has 9 people, each part-time.

| Role | People | Effort (PM) | Share | Basis |
|---|---|---|---|---|
| Developers | 3 | 7.0 | 44% | Construction is the largest activity |
| Tester / QA | 2 | 3.0 | 19% | About 20% of effort goes to testing |
| Project Manager / Scrum Master | 1 | 1.5 | 9% | Planning, sprint events, tracking |
| System Analyst / Product Owner | 1 | 1.5 | 9% | Requirements, backlog, UML |
| UI/UX Designer | 1 | 1.5 | 9% | Wireframes and prototype |
| Database Engineer | 1 | 1.5 | 9% | Schema, queries, data import |
| **Total** | **9** | **16.0** | **100%** | |

## 6.5 How the Number of Developers and Testers Was Decided

- **Developers:** 7.0 PM over 3.7 months is about 1.9 full-time developers. Three part-time developers are used so that three work streams can run in parallel: student features, advisor/admin features, and integrations (payment, e-mail).
- **Testers:** 3.0 PM over 3.7 months is about 0.8 full-time testers. Two part-time testers are used: one for functional and regression testing, one for performance, security and user acceptance testing. The developer-to-tester ratio is 3 : 2.
- **Capacity check:** six delivery sprints at about 20 story points give 120 points of capacity against a 102-point backlog.

## 6.6 Project Cost Estimation

| Cost component | Basis (assumed) | Estimated cost (BDT) |
|---|---|---|
| Developers | 7.0 PM × ৳60,000 | ৳420,000 |
| Tester / QA | 3.0 PM × ৳45,000 | ৳135,000 |
| Project Manager / Scrum Master | 1.5 PM × ৳80,000 | ৳120,000 |
| System Analyst | 1.5 PM × ৳70,000 | ৳105,000 |
| UI/UX Designer | 1.5 PM × ৳50,000 | ৳75,000 |
| Database Engineer | 1.5 PM × ৳65,000 | ৳97,500 |
| Software / Tools | Issue tracker, Git hosting, design and API-testing tools, plug-in licences | ৳20,000 |
| Server / Hosting | VPS for development, staging and production: 4 months × ৳4,000 | ৳16,000 |
| Domain | One year; TLS certificate is free (Let's Encrypt) | ৳1,500 |
| Hardware / Equipment | Allocated cost of test devices and networking | ৳30,000 |
| Other expenses | Internet and utilities ৳20,000 + user manual and training ৳10,000 + 10% contingency on direct cost (৳1,050,000 × 10% = ৳105,000) | ৳135,000 |
| **Total estimated cost** | | **৳1,155,000** |

Personnel is ৳952,500, or 82% of the total. The cost per use case point is about ৳9,040. The monthly salaries are assumed market rates for Bangladesh and can be changed in the estimation sheet.

## 6.7 Assumptions

1. One person-month = 160 working hours; the working week is Sunday–Thursday.
2. Productivity is 20 person-hours per UCP (Karner), and the ratings in the TCF and ECF tables are the team's judgement.
3. The 16-week duration is an assumed time-box; the real duration is set by the Faculty.

4. Effort covers analysis, design, coding, testing, project management and documentation.
5. There is no significant reuse of existing code; open-source frameworks and free-tier tools are used.
6. All 9 specialists are part-time and are paid only for the effort they contribute.
7. The student group of 3 shares these roles (Chapter 7); the figures describe a professional team building the same system.

### 6.8 What If the Duration or the Requirements Change?

- **Longer duration:** personnel cost depends on effort, not calendar time, but each extra week with the full team adds about ৳60,000 (৳952,500 ÷ 16 weeks) plus about ৳4,000 for hosting and internet. Adding people to a late project adds communication overhead, so the contingency and Sprint 6 absorb small delays first.
- **Requirement change:** every 10 extra UCP adds 10 × 1.035 × 0.98 × 20 ≈ 203 hours (1.3 PM, about ৳75,000). The change enters the Product Backlog and is estimated in story points before it is accepted.

## Chapter 7 – Project Planning

### 7.1 Schedule

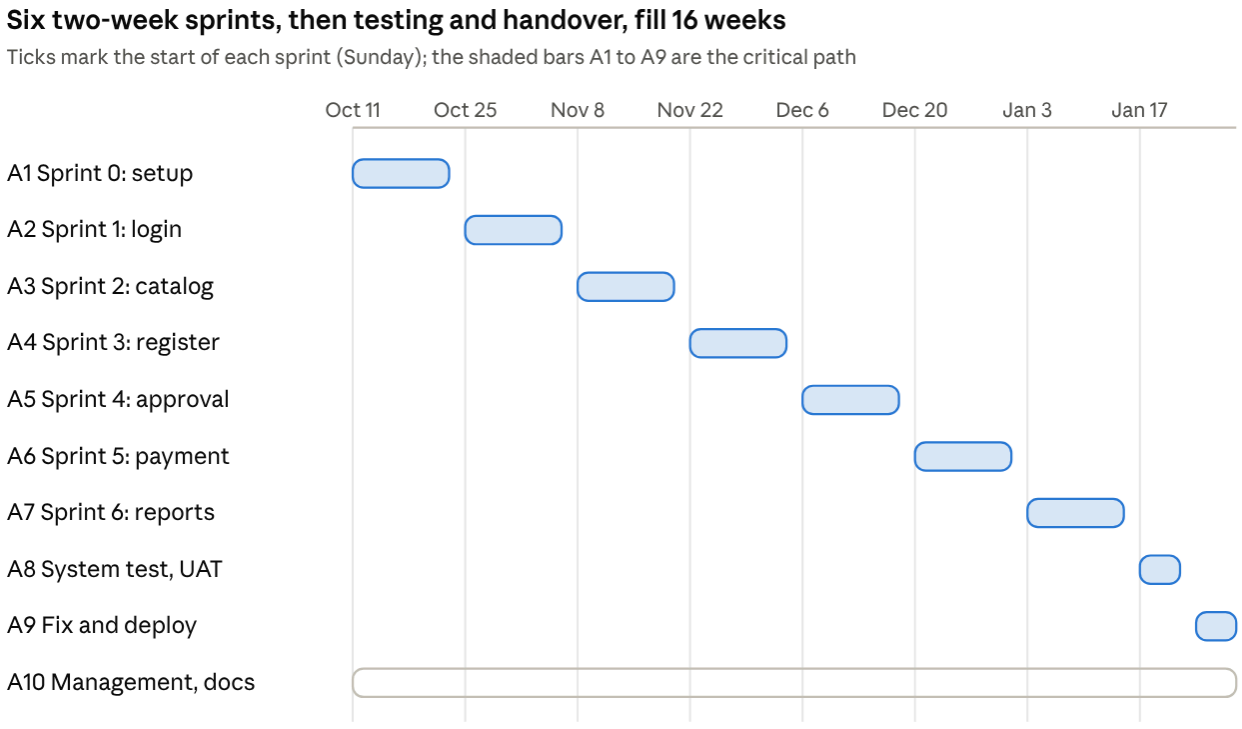
The project runs for 16 weeks from Sunday 11 October 2026 to Thursday 28 January 2027 (Sunday–Thursday working weeks). Public holidays are not scheduled; the start date is an assumption to be replaced by the Faculty's dates.

| ID | Activity | Duration | Start | End | Responsible | Depends on |
|---|---|---|---|---|---|---|
| A1 | Sprint 0: requirements, backlog, architecture, database schema, wireframes | 2 weeks | 11 Oct 2026 | 22 Oct 2026 | Analyst/PO, UI/UX, DB engineer, Scrum Master | – |
| A2 | Sprint 1: login and profile (PB01, PB02) | 2 weeks | 25 Oct 2026 | 5 Nov 2026 | Developers, QA | A1 |
| A3 | Sprint 2: catalog, offerings, search (PB03–PB05) | 2 weeks | 8 Nov 2026 | 19 Nov 2026 | Developers, DB engineer, QA | A2 |
| A4 | Sprint 3: registration with validation, schedule (PB06, PB07) | 2 weeks | 22 Nov 2026 | 3 Dec 2026 | Developers, QA | A3 |
| A5 | Sprint 4: approval, drop, waitlist (PB08–PB10) | 2 weeks | 6 Dec 2026 | 17 Dec 2026 | Developers, QA | A4 |

| ID | Activity | Duration | Start | End | Responsible | Depends on |
|---|---|---|---|---|---|---|
| A6 | Sprint 5: payment, notifications, slip (PB11–PB13) | 2 weeks | 20 Dec 2026 | 31 Dec 2026 | Developers, QA | A5 |
| A7 | Sprint 6: users, reports, hardening (PB14, PB15) | 2 weeks | 3 Jan 2027 | 14 Jan 2027 | Developers, DB engineer, QA | A6 |
| A8 | System testing and user acceptance testing | 1 week | 17 Jan 2027 | 21 Jan 2027 | QA, Product Owner | A7 |
| A9 | Bug fixing, deployment, handover, documentation | 1 week | 24 Jan 2027 | 28 Jan 2027 | Developers, Scrum Master | A8 |
| A10 | Project management, Scrum events and documentation | 16 weeks | 11 Oct 2026 | 28 Jan 2027 | Scrum Master | continuous |

## 7.2 Gantt Chart

The Gantt chart below is drawn from the schedule table; the critical path runs through A1 → A9.




Gantt chart · 10 activities, 16 weeks

A10 (project management and documentation) runs alongside the whole project; Section 7.1 gives the full activity names, owners and dependencies.

### 7.3 Process Model and Backlog

The selected process model is Scrum. The Product Backlog (15 items, 102 story points) and the Sprint Plan are in Sections 3.5 and 3.6.

### 7.4 Team Responsibilities

The group has three members. Each member holds one lead role and one supporting role, and every member also writes code and tests. Names are to be filled in by the group.

| Member | Lead role | Supporting role | Main artifacts |
|---|---|---|---|
| [Member 1] | Product Owner, System Analyst, UI/UX Designer | Developer (front end) | Problem statement, requirements, use case and activity diagrams, prototype |
| [Member 2] | Scrum Master, Database Engineer | Developer (back end) | Process model, backlog and sprint plan, class diagram, database, estimation, cost, Gantt chart |
| [Member 3] | Tester / QA | Developer (integrations) | Sequence diagrams, test cases, risk analysis, traceability |

Cross-understanding rule: before the viva each member reviews the artifacts of the other two members and prepares answers to the questions in the Appendix.

## Chapter 8 – Testing

### 8.1 Test Strategy

| Level | Scope | Performed by | When |
|---|---|---|---|
| Unit testing | Validation rules (prerequisite, time conflict, seat and credit limit), invoice calculation, status changes | Developers | Inside every sprint |
| Integration testing | Controller + database, payment gateway sandbox, e-mail service, waitlist promotion | Developers and QA | End of each sprint |

| Level | Scope | Performed by | When |
|---|---|---|---|
| System testing | Complete workflows and non-functional requirements (performance, security, concurrency) | QA | Release phase (A8) and after each sprint (regression) |
| User acceptance testing (UAT) | Real scenarios run by sample students, an advisor and an admin staff member | Product Owner and users | Release phase (A8) |

Entry criterion: the story meets the Definition of Done. Exit criterion: all Must-priority test cases pass and no open defect of Critical or High severity remains. Defects are ranked Critical (data loss, over-booked seat, security hole), High (feature unusable), Medium (workaround exists) and Low (cosmetic).

## 8.2 Test Cases

| ID | Level | Req. | Use case | Scenario and input | Expected result |
|---|---|---|---|---|---|
| TC01 | System | FR01 | UC01 | Log in with a valid student ID and password | Student dashboard opens |
| TC02 | System | FR01, NFR02 | UC01 | Enter a wrong password 5 times in a row | An error appears each time; the account is locked for 15 minutes after the 5th |
| TC03 | Unit | FR05 | UC15 | Register for a course whose prerequisite is not completed | Registration rejected; the missing prerequisite is listed |
| TC04 | Unit | FR06 | UC16 | Register for a section that overlaps a registered course (same day, overlapping time) | Registration rejected; the conflicting course is shown |
| TC05 | Unit | FR07 | UC17 | Register for a section with no free seat (capacity reached) | Registration rejected; waitlist option is offered |
| TC06 | Unit | FR07 | UC17 | Student has 16 credits and adds a 3-credit course (limit 18) | Registration rejected; credit-limit message |
| TC07 | Integration | FR04, FR15 | UC04, UC18 | Register for a valid open section | Registration saved as Pending; seat count reduced by 1; student and advisor notified |
| TC08 | System | FR04 | UC04 | Register after the registration window has closed | Rejected with 'registration period over' |

| ID | Level | Req. | Use case | Scenario and input | Expected result |
|---|---|---|---|---|---|
| TC09 | Integration | FR08 | UC06 | Join the waitlist of a full section, then another student drops | First waitlisted student is registered automatically and notified |
| TC10 | System | FR09 | UC05 | Drop a course before the deadline | Status Dropped; seat released; invoice recalculated |
| TC11 | System | FR09 | UC05 | Drop a course after the deadline | Drop refused with the deadline shown |
| TC12 | System | FR10, FR15 | UC09 | Advisor approves a Pending registration | Status Approved; student notified |
| TC13 | System | FR10 | UC09 | Advisor rejects without a remark | Action blocked; remark is required |
| TC14 | System | FR10, NFR02 | UC09 | Advisor opens the registration of a student who is not an advisee (by changing the URL) | Access denied (HTTP 403) |
| TC15 | Integration | FR11, FR15 | UC08 | Pay an invoice in the gateway sandbox (success) | Invoice marked Paid; confirmation sent |
| TC16 | Integration | FR11 | UC08 | Gateway returns a failure | Invoice stays Unpaid; student can retry |
| TC17 | System | FR13 | UC11 | Admin creates an offering with capacity 0 | Rejected: capacity must be greater than 0 |
| TC18 | System | FR14 | UC12 | Admin disables a user; the user tries to log in | Login refused |
| TC19 | System | FR12, FR16 | UC07, UC14 | Open the schedule and print the registration slip | Timetable shows all registered sections; PDF lists approved courses and payment status |
| TC20 | System | NFR01, NFR07 | UC03, UC04 | 500 concurrent users search and register | 95% of requests complete within 3 seconds |
| TC21 | System | NFR05 | UC04 | 50 students register for the last seat at the same moment | Exactly one succeeds; the seat count never goes below 0 |
| TC22 | System | NFR02 | – | Open an admin URL with a student session | Access denied; no data shown |
| TC23 | UAT | FR03, NFR04 | UC03, UC04 | Sample students search by code, title and instructor, then register | Correct results; 90% finish within 5 minutes |
| TC24 | UAT | FR17 | UC13 | Admin generates the enrollment report for a semester and exports CSV | Totals match the database; the CSV opens correctly |
| TC25 | System | FR02 | UC02 | Change the password with a wrong old password, then with the correct one | Wrong old password refused; correct one saves and the new password works |

| ID | Level | Req. | Use case | Scenario and input | Expected result |
|---|---|---|---|---|---|
| TC26 | System | FR13 | UC10 | Admin creates a course with a course code that already exists | Rejected: course code must be unique |

Test execution results (Pass/Fail, date, tester) are recorded in the testing document for each sprint.

# Chapter 9 – Risk Analysis

The highest-exposure risk is seat over-booking when many students register at once (R1); Sprint 3 is therefore the highest-risk sprint. Probability and impact are scored Low = 1, Medium = 2, High = 3; exposure = probability × impact.

| ID | Risk | Probability | Impact | Exposure | Mitigation | Owner |
|---|---|---|---|---|---|---|
| R1 | Seat over-booking when many students register simultaneously | Medium (2) | High (3) | 6 | Database transactions with row locking on the seat count; concurrency test TC21 in Sprint 3 | Back-end developer |
| R2 | Requirements change mid-project (credit limit, waitlist or drop policy) | High (3) | Medium (2) | 6 | Scrum backlog re-ordering; business rules kept in a configuration table; Product Owner review every sprint | Product Owner |
| R3 | Payment gateway integration delayed or sandbox unavailable | Medium (2) | High (3) | 6 | Payment accessed through an interface with a mock implementation; integration spike early in Sprint 4; offline payment as fallback | Integration developer |
| R4 | Slow response during the registration rush | Medium (2) | High (3) | 6 | Database indexes, cached course catalog, load test TC20, scalable hosting | Back-end developer, QA |
| R5 | Team member unavailable (exams, illness, part-time work) | High (3) | Medium (2) | 6 | Cross-training so every member knows the whole system; pair work; buffer in Sprint 6 | Scrum Master |

| ID | Risk | Probability | Impact | Exposure | Mitigation | Owner |
|---|---|---|---|---|---|---|
| R6 | Unauthorized access or data leak | Low (1) | High (3) | 3 | Role-based access, hashed passwords, HTTPS, input validation; security tests TC14 and TC22 | Back-end developer |
| R7 | Effort estimate is inaccurate | Medium (2) | Medium (2) | 4 | Re-estimate after Sprint 1 using real velocity; 10% cost contingency | Scrum Master |
| R8 | Student completed-course data is wrong or missing, so prerequisite checks fail | Medium (2) | Medium (2) | 4 | CSV import with validation; admin correction screen | Database engineer |
| R9 | Scope creep | Medium (2) | Medium (2) | 4 | Scope fixed in Section 1.7; every change goes through the Product Owner and, for major change, the Faculty | Product Owner |

# Chapter 10 – Traceability

Every functional requirement is traced forward to a use case, an activity or sequence diagram where one exists, the classes involved, a prototype screen and at least one test case. AD1 and SD1 cover registration, AD1 and SD2 cover approval, and AD2 covers payment; the remaining requirements are simple data-maintenance flows that do not need their own diagram.

## 10.1 Requirement-to-Design and Requirement-to-Test

| Requirement | Use case | Activity | Sequence | Classes | Prototype | Test cases |
|---|---|---|---|---|---|---|
| FR01 | UC01 | – | – | User | S01 | TC01, TC02 |
| FR02 | UC02 | – | – | User | S02 | TC25 |
| FR03 | UC03 | AD1 | – | Course, CourseOffering, Department | S04 | TC23 |
| FR04 | UC04 | AD1 | SD1 | Student, Registration, CourseOffering | S05 | TC07, TC08, TC23 |
| FR05 | UC15 | AD1 | SD1 | Course, Student | S05 | TC03 |
| FR06 | UC16 | AD1 | SD1 | CourseOffering, Registration | S05 | TC04 |

| Requirement | Use case | Activity | Sequence | Classes | Prototype | Test cases |
|---|---|---|---|---|---|---|
| FR07 | UC17 | AD1 | SD1 | CourseOffering, Student | S05 | TC05, TC06, TC21 |
| FR08 | UC06 | AD1 | – | Registration, CourseOffering | S06 | TC09 |
| FR09 | UC05 | – | – | Registration, CourseOffering | S08 | TC10, TC11 |
| FR10 | UC09 | AD1 | SD2 | FacultyAdvisor, Registration | S10 | TC12, TC13, TC14 |
| FR11 | UC08 | AD2 | – | Invoice, Payment | S09 | TC15, TC16 |
| FR12 | UC07 | – | – | Student, Registration | S03, S07 | TC19 |
| FR13 | UC10, UC11 | – | – | Administrator, Course, CourseOffering, Department | S11, S12 | TC17, TC26 |
| FR14 | UC12 | – | – | Administrator, User | S13 | TC18 |
| FR15 | UC18 | AD1, AD2 | SD1, SD2 | Notification | S16 | TC07, TC12, TC15 |
| FR16 | UC14 | – | – | Registration, Invoice | S15 | TC19 |
| FR17 | UC13 | – | – | Administrator | S14 | TC24 |

## 10.2 Non-Functional Requirements

| Requirement | Design decision | Verified by |
|---|---|---|
| NFR01 Performance | Indexed queries, cached course catalog, stateless web tier | TC20 |
| NFR02 Security | Hashed password (User.passwordHash), role checks on every request, HTTPS | TC02, TC14, TC22 |
| NFR03 Availability | Health checks, staging environment, backups | Uptime monitoring during UAT (A8) |
| NFR04 Usability | Responsive layout, inline validation messages (S05) | TC23 |
| NFR05 Reliability | Transactional seat reservation (CourseOffering.reserveSeat) | TC21 |
| NFR06 Maintainability | Layered architecture, unit tests for business rules | Unit-test coverage report (target 70%) |
| NFR07 Scalability | Stateless application tier that can be replicated | TC20 |

### 10.3 Consistency Analysis

- All 17 functional requirements map to at least one use case and at least one test case; all 7 non-functional requirements map to a design decision and a verification.
- All 14 user-goal use cases appear in the use case diagram, in at least one prototype screen and in at least one test case. UC15–UC17 are shown on screen S05 as validation messages and are tested by TC03–TC06; UC18 is tested by TC07, TC12 and TC15.
- The same actors (Student, Faculty Advisor, Administrator, Payment Gateway, Notification Service) appear in the use case diagram, activity diagrams and sequence diagrams.
- Alternate flows are consistent: a failed validation ends registration in UC04, AD1 and SD1; a Reject releases the seat in UC09, AD1 and SD2.
- Domain messages in the sequence diagrams (reserveSeat, submit, approve, reject, releaseSeat, send) exist as operations in the class diagram.
- Artifact counts: 17 FR, 7 NFR, 18 use cases, 2 activity diagrams, 2 sequence diagrams, 11 classes, 16 screens, 26 test cases.

## Chapter 11 – Conclusion

### 11.1 Summary

The project analysed the manual course registration problem and specified an Online Course Registration System with 17 functional and 7 non-functional requirements. It was designed with UML, planned with Scrum (15 backlog items, 102 story points, six delivery sprints), sized with Use Case Points (127.8 UCP, 16.0 person-months), costed at ৳1,155,000, and covered by 26 test cases, 9 risks and a full traceability matrix.

### 11.2 Learning Outcomes

- Turning a real problem into testable requirements, actors and use cases.
- Modelling structure and behaviour with consistent UML diagrams.
- Applying a process model (Scrum) with a real backlog and sprint plan instead of only describing it.
- Estimating size, effort, time, team and cost with a formal technique and stated assumptions.
- Linking requirements, design, prototype and tests through traceability.

### 11.3 Limitations

- Grades, transcripts and CGPA are out of scope; prerequisite checks rely on imported completed-course data.
- Only a web interface is provided; there is no native mobile app.
- The estimate relies on the team's own factor ratings and assumed salaries, so it is an approximation.
- The prototype demonstrates the workflows but is not a production system.

### 11.4 Future Improvements

- Integration with the university result system for automatic prerequisite data.
- Native mobile app and push notifications.
- Timetable suggestions that avoid clashes automatically and priority rules for waitlists.
- Analytics dashboards on seat demand to help plan future offerings.

## Appendix – Viva Preparation

Short answers to the handbook's viva questions, with the section to read for detail.

| Question | Short answer | Section |
|---|---|---|
| Why did you select this process model? | Scrum fits feature-by-feature delivery, policy changes every semester and early user feedback; the riskiest sprint is built early | 3.2 |
| Why is this actor required? | Student registers, Advisor approves (a university rule), Admin maintains data; the gateway and notification service are external systems the system depends on | 2.1 |
| Why include or extend? | Include = mandatory shared steps (checks, notification); extend = optional or conditional behaviour (waitlist when full, print slip) | 4.1 |
| Why is this class required? | Each class supports specific use cases; see the class table | 4.4 |

| Question | Short answer | Section |
|---|---|---|
| How is the sequence diagram related to the use case diagram? | SD1 realizes the main and alternate flows of UC04, and SD2 realizes UC09, with the same actors and the same failure paths | 4.3, 10.3 |
| How did you estimate developers and testers? | Role effort ÷ project duration gives about 1.9 developers and 0.8 testers, rounded to 3 and 2 part-time people | 6.5 |
| How did you calculate the cost? | Effort per role × assumed monthly salary, plus tools, hosting, domain, hardware, other items and 10% contingency | 6.6 |
| What assumptions did you make? | 160 hours per person-month, 20 hours per UCP, 16-week time-box, part-time team, no major reuse | 6.7 |
| What if the duration increases? | About ৳60,000 per extra week with the full team; contingency and Sprint 6 absorb small delays | 6.8 |
| What if requirements change? | The change enters the backlog; 10 extra UCP is about 203 hours or ৳75,000; business rules are configurable | 6.8, R2 |
| What is the highest-risk part? | Seat allocation under concurrent registration (R1, UC04, Sprint 3) | Chapter 9 |
| How would another process model change the project? | With Waterfall, requirements would be frozen in the first weeks, testing would start only after coding, and the seat-concurrency defect would be found late | 3.3 |
| Can you explain the effort calculation? | UCP = 126 × 1.035 × 0.98 = 127.8; effort = 127.8 × 20 = 2,556 h; 2,556 ÷ 160 = 16.0 PM | 6.2, 6.3 |